



HOTEL RECOMMENDATION SYSTEM

TEAM AND ROLES

PREPROCESSING = ABHISHEK [12305620]

MODEL CODING = KARUNA [12315451]

UJJWAL KUMAR [12316568]

GUI CODING = SARTHAK MATLOTIA [12316277]

AYUSH LUTHRA [12322358]



What is Hotel Recommendation System?

A hotel recommendation system is an AI-based tool that helps users find the best hotels based on their preferences. It uses various algorithms at backend to analyze data such as user ratings, reviews, location, price, and amenities to suggest hotels that meet the user's criteria. These systems can be integrated into travel websites, mobile apps, or used as standalone applications. They can also be personalized based on the user's past booking history and preferences.

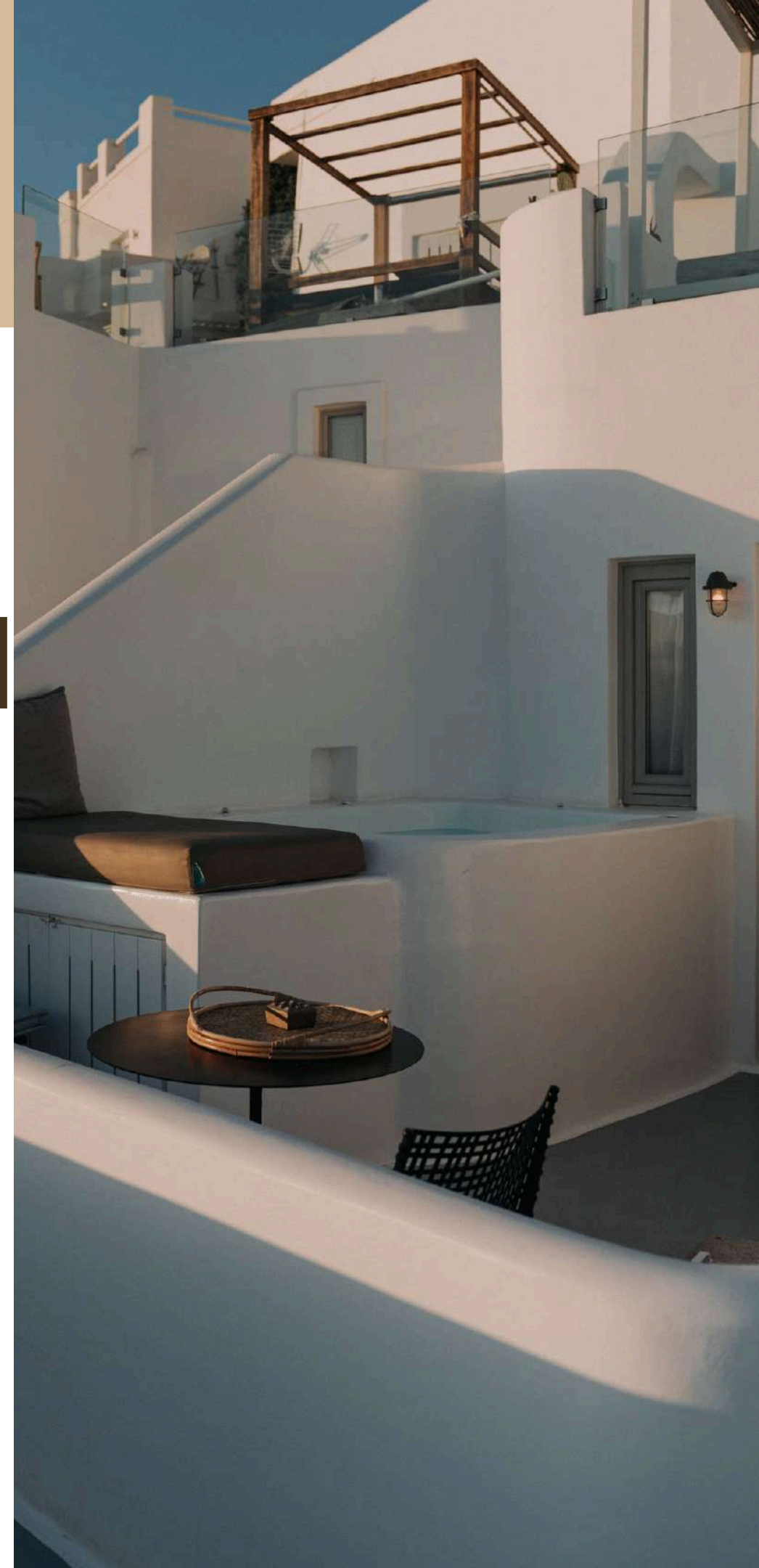
DATA COLLECTION

Data Collection Tool = KAGGLE

Dataset Name = Hotels In India Dataset [2023]

ATTRIBUTES NAME

- Hotel_id
- State
- City
- Ratings
- Alcohol Availability
- Hotel Name
- Total Rooms
- Category
- Pricing
- Meals

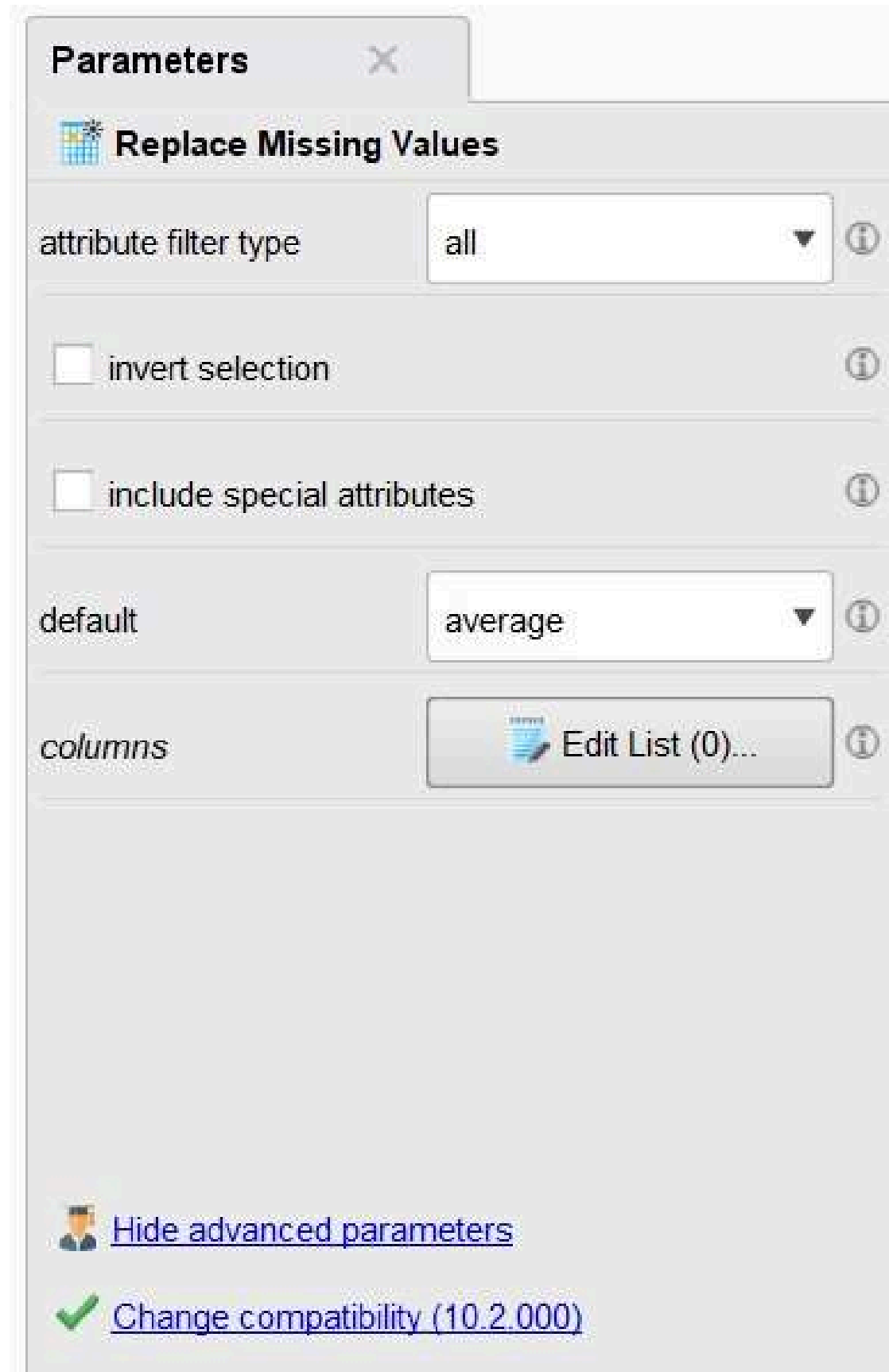


Data Preprocessing

Replace Missing Value

**This Operator replaces missing values in
Examples of selected Attributes by a
specified replacement.**

Here we have replaced the missing values
with average values



The screenshot shows the 'Parameters' window for the 'Replace Missing Values' operator. The window has a title bar with a close button. Below the title bar, there is a section header 'Replace Missing Values' with a small icon. The parameters are organized into sections: 'attribute filter type' with a dropdown menu set to 'all'; 'invert selection' with an unchecked checkbox; 'include special attributes' with an unchecked checkbox; 'default' with a dropdown menu set to 'average'; and 'columns' with a button labeled 'Edit List (0)...'. At the bottom, there are two links: 'Hide advanced parameters' and 'Change compatibility (10.2.000)'.

Parameters

 **Replace Missing Values**

attribute filter type: all

☐ invert selection

☐ include special attributes

default: average

columns:  Edit List (0)...

 [Hide advanced parameters](#)

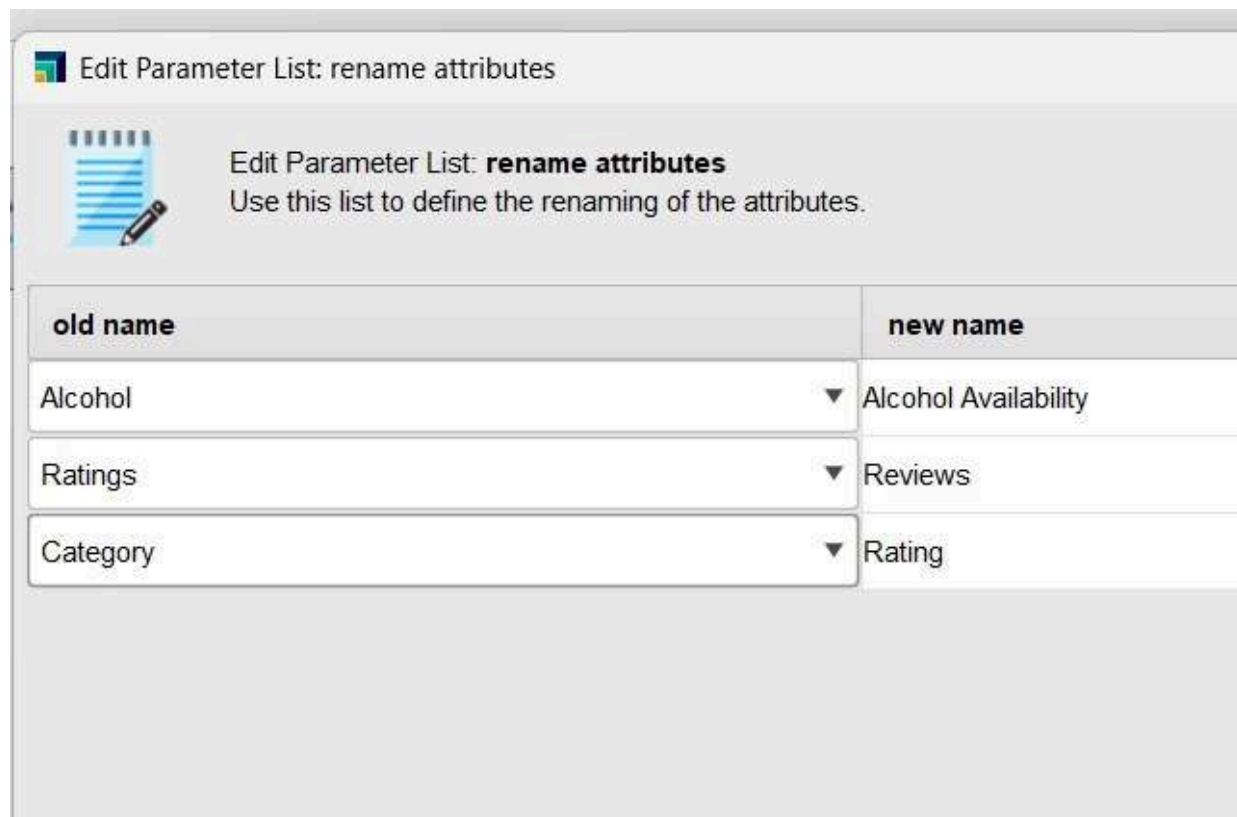
 [Change compatibility \(10.2.000\)](#)

Data Preprocessing

Rename

This operator can be used to rename one or more attributes of an ExampleSet.

Here we have changed the Alcohol
to Alcohol Availability
Rating to Reviews and Category to
Rating



Edit Parameter List: rename attributes

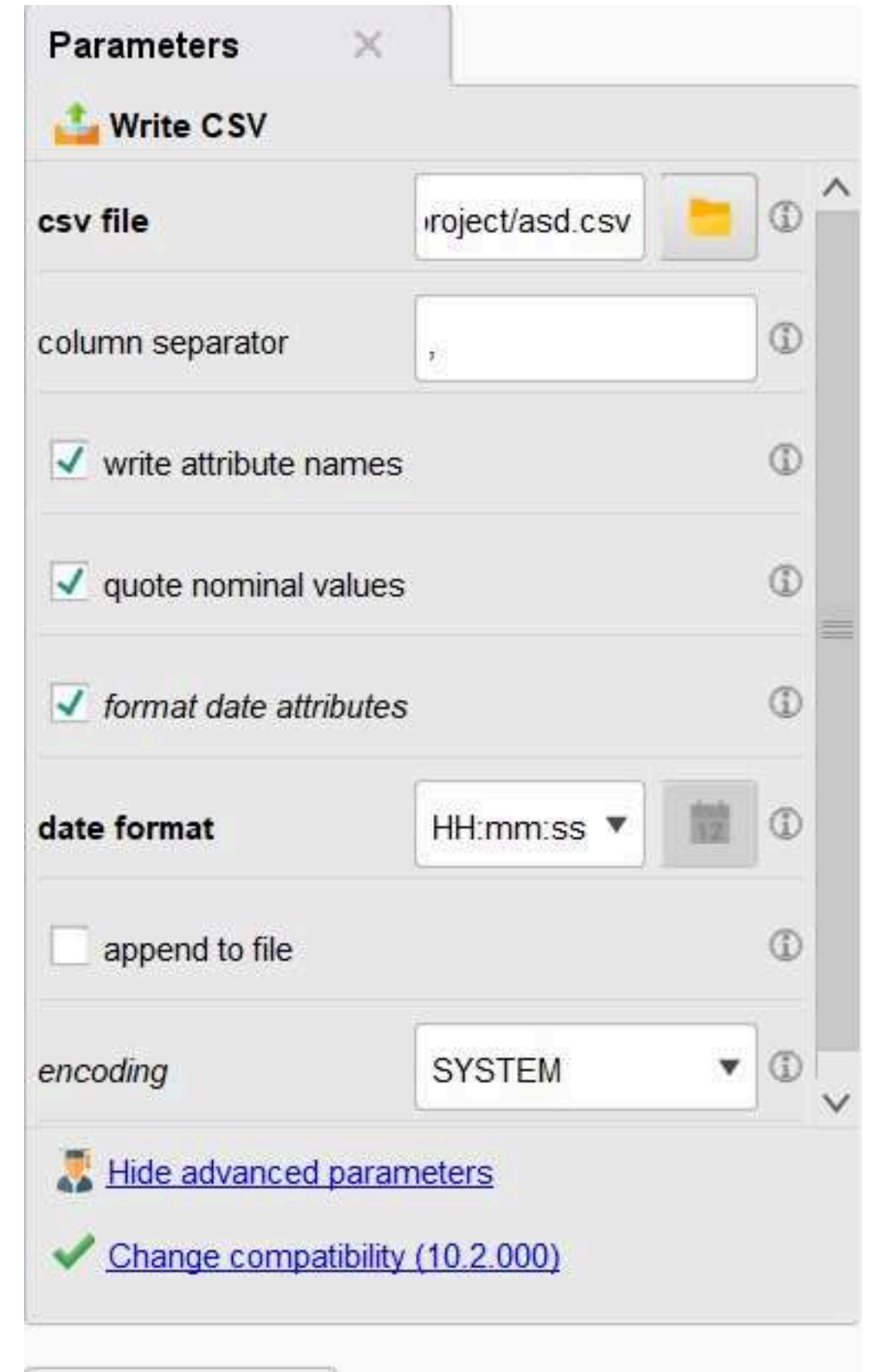
Edit Parameter List: **rename attributes**
Use this list to define the renaming of the attributes.

old name		new name
Alcohol	▼	Alcohol Availability
Ratings	▼	Reviews
Category	▼	Rating

Write CSV

This operator is used to write CSV files(Comma-Separated Values).

This operator we have
used to store the pre-
processed csv file locally
on our system



Parameters

Write CSV

csv file: project/asd.csv

column separator: ,

☒ write attribute names

☒ quote nominal values

☒ format date attributes

date format: HH:mm:ss

☐ append to file

encoding: SYSTEM

[Hide advanced parameters](#)

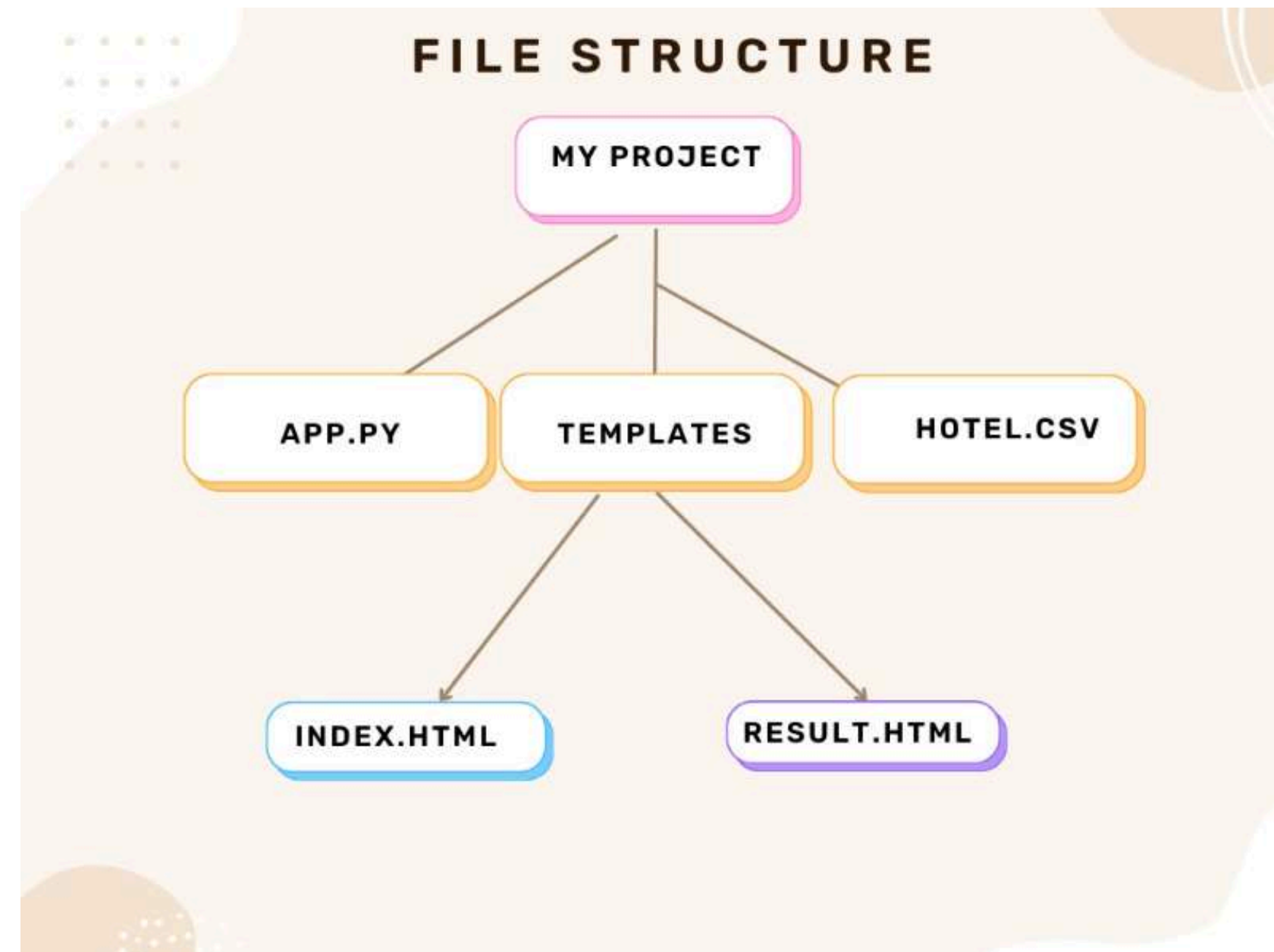
[Change compatibility \(10.2.000\)](#)

Data Preprocessing

This Code here is used to clean the data by removing special characters for it

```
Sub ReplaceSpecial()  
    Dim cel As Range  
    Dim strVal As String  
    Dim i As Long  
    Application.ScreenUpdating = False  
    For Each cel In Selection  
        strVal = cel.Value  
        For i = 1 To Len(strVal)  
            Select Case Asc(Mid(strVal, i, 1))  
                Case 32, 48 To 57, 65 To 90, 97 To 122  
                    ' Leave ordinary characters alone  
                Case Else  
                    Mid(strVal, i, 1) = " "  
            End Select  
        Next i  
        cel.Value = strVal  
    Next cel  
    Application.ScreenUpdating = True  
End Sub
```

PROJECT STRUCTURE



App.js

app.py X

C: > Users > DELL5 > hotel_recommend > app.py > recommend_hotels

```
1  from flask import Flask, render_template, request
2  import pandas as pd
3  from sklearn.model_selection import train_test_split
4
5  app = Flask(__name__)
6
7  # Load the dataset and prepare it
8  df_hd = pd.read_csv("hotels.csv")
9  hotel_details = df_hd.iloc[:, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]]
10
11 # Assume hotel_details is already prepared
12 X = hotel_details[['hotel_id', 'state', 'city', 'ratings', 'alcohol_availability', 'hotel_name', 'total_rooms', 'category', 'pricing', 'meals']]
13 y = hotel_details['hotel_name']
14 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.26, random_state=42)
15
16 @app.route('/')
17 def index():
18     return render_template('index.html')
19
20 @app.route('/recommend_hotels', methods=['POST'])
21 def recommend_hotels():
22     # Prompt user for input
23     user_input = {
24         'state': request.form['state'].lower(),
25         'city': request.form['city'].lower(),
26         'ratings': float(request.form['ratings']),
27         'min_pricing': float(request.form['min_pricing']),
28         'max_pricing': float(request.form['max_pricing'])
29     }
30
31     meals_option = request.form['meals_option']
32
33     # Convert state and city to lowercase for case-insensitive comparison
34     X_test['state'] = X_test['state'].str.lower()
35     X_test['city'] = X_test['city'].str.lower()
```

```

36
37 # Apply filtering conditions
38 state_filter = X_test['state'] == user_input['state']
39 city_filter = X_test['city'] == user_input['city']
40 ratings_filter = X_test['ratings'] >= user_input['ratings']
41 pricing_filter = (X_test['pricing'] >= user_input['min_pricing']) & (X_test['pricing'] <= user_input['max_pricing'])
42
43 # Filtering by Meals Option
44 if meals_option == '1':
45     meals_filter = True # No filtering based on meals
46 elif meals_option == '2':
47     meals_filter = X_test['meals'].str.contains('breakfast', case=False)
48 elif meals_option == '3':
49     meals_filter = X_test['meals'].str.contains('lunch', case=False)
50 elif meals_option == '4':
51     meals_filter = X_test['meals'].str.contains('dinner', case=False)
52 else:
53     print("Invalid option selected. Defaulting to 'All meals'.")
54     meals_filter = True
55
56 # Apply all filtering conditions
57 filtered_data = X_test[state_filter & city_filter & ratings_filter & pricing_filter & meals_filter]
58
59 if not filtered_data.empty:
60     recommended_hotels = filtered_data['hotel_id'].tolist() # Get recommended hotel IDs
61     recommended_hotels_names = [y_test.loc[X_test[X_test['hotel_id'] == hotel_id].index[0]] for hotel_id in recommended_hotels] # Get hotel names
62     return render_template('result.html', recommended_hotels=recommended_hotels_names)
63 else:
64     return "No hotels match the specified criteria."
65
66 if __name__ == '__main__':
67     app.run(debug=True)
68

```


INDEX.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Hotel Recommendation</title>
7   <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
8 </head>
9 <body>
10   <h1>Hotel Recommendation</h1>
11   <form action="/recommend_hotels" method="post">
12     <label for="state">State:</label>
13     <input type="text" id="state" name="state" required><br><br>
14
15     <label for="city">City:</label>
16     <input type="text" id="city" name="city" required><br><br>
17
18     <label for="ratings">Minimum Ratings:</label>
19     <input type="number" id="ratings" name="ratings" step="0.01" required><br><br>
20
21     <label for="min_pricing">Minimum Pricing:</label>
22     <input type="number" id="min_pricing" name="min_pricing" step="0.01" required><br><br>
23
24     <label for="max_pricing">Maximum Pricing:</label>
25     <input type="number" id="max_pricing" name="max_pricing" step="0.01" required><br><br>
26
27     <label for="meals_option">Select Meals Option:</label>
28     <select id="meals_option" name="meals_option" required>
29       <option value="all_meals">All meals</option>
30       <option value="breakfast">Breakfast</option>
31       <option value="lunch">Lunch</option>
32       <option value="dinner">Dinner</option>
33     </select><br><br>
34
35     <input type="submit" value="Submit">
36   </form>
37 </body>
38 </html>
39
```

RESULT.html

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Hotel Recommendation Result</title>
7  </head>
8  <body>
9      <h1>Recommended Hotels</h1>
10     <ul>
11         {% for hotel_name in recommended_hotels %}
12         <li>{{ hotel_name }}</li>
13         {% endfor %}
14     </ul>
15 </body>
16 </html>
```


OUTPUT

Hotel Recommendation System

State:

City:

Minimum Ratings:

Minimum Pricing:

Maximum Pricing:

Select Meals Option:

Hotel Recommendation System

State:

City:

Minimum Ratings:

Minimum Pricing:

Maximum Pricing:

Select Meals Option:

Recommended Hotels

- HOTEL NALANDA
- Gulmohar Greens Golf Country Club Limited
- BINORI HOTEL PVT LTD
- AJU IMPERIAL HOTEL A UNIT OF AJU RYOKAN NCR PVT LTD

*Thank
you!*

